



Ayudantía 1

1. Diseñe una máquina de Turing que acepte el siguiente lenguaje:

$$L = \{a^{2^n} \in \{a\}^* \mid n \geq 0\}$$

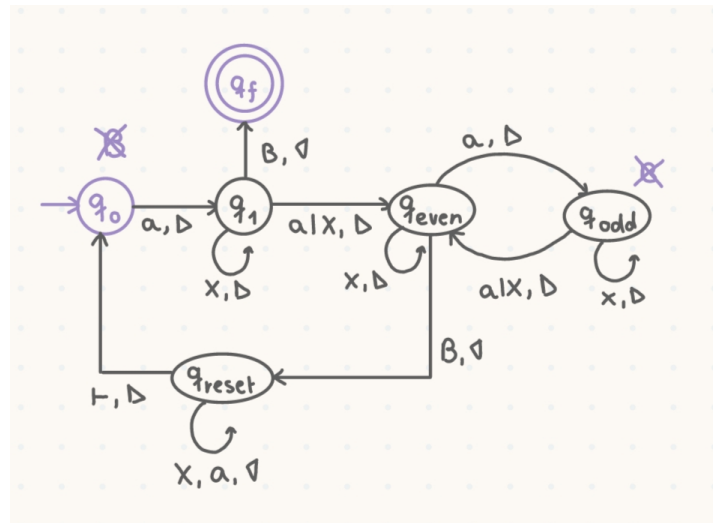
Entregue una descripción formal de la máquina (alfabeto y diagrama de estados) junto con un pseudocódigo.

Solución: Se puede observar que las potencias de dos corresponden exactamente a aquellos número que al dividirse reiteradamente entre 2 hasta llegar a un número impar, dan 1. Por ejemplo como 16 es potencia de dos al realizar este procedimiento obtenemos $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$, mientras que en el caso de un número que no es una potencia de 2 obtenemos un número impar distinto a 1 como resultado de estas divisiones, por ejemplo $20 \rightarrow 10 \rightarrow 5$. Con estas nociones, definimos el siguiente pseudocódigo.

-
1. Si la entrada es vacía, **rechazar**.
 2. Mover la cabeza lectora hacia la derecha, tachando cada segundo símbolo «a» encontrado.
 3. Si en el paso (2), la cabeza lectora vio solamente un símbolo «a», **aceptar**.
 4. Si en el paso (2), la cabeza lectora vio un número impar de símbolos «a», **rechazar**.
 5. Mover la cabeza lectora al inicio y repetir el paso (2).
-

Nótese que, en cada iteración del paso (2), si la cantidad de símbolos «a» existentes es par, esta cantidad se reduce a la mitad.

Considerando lo anterior, definimos una máquina de Turing con alfabeto $\Gamma = \{a, X, \vdash, B, \}$, donde el símbolo «X» nos permitirá eliminar las letras revisadas. El diagrama de estado de esta máquina es el siguiente:



La máquina comienza en el estado inicial q_0 y transiciona al estado q_1 cuando la cabeza lectora ve el primer símbolo «a» (si la palabra es vacía, la máquina no logra salir de q_0 , la ejecución se termina y rechaza). Si la cabeza lectora no ve más símbolos «a» estando en q_1 (ya no quedan más letras por revisar) la máquina cambia su estado a q_f estado final y acepta, puesto que la palabra vista estaba compuesta por un único símbolo «a».

En caso contrario, la cabeza lectora comienza a eliminar todos los segundos símbolos «a» vistos, alternando entre q_{even} , estado de equilibrio (misma cantidad de símbolos «a» vistos y tachados, cantidad par de símbolos) y q_{odd} , estado de desequilibrio (la cabeza lectora ha visto un símbolo «a» más de los que ha tachado, cantidad impar de símbolos). Así, si la palabra se acaba en q_{odd} la máquina se detiene y rechaza, mientras que si la palabra se acaba en q_{even} la máquina transiciona a q_{reset} , estado que devuelve la cabeza lectora al inicio de la palabra para repetir el proceso. Nótese que los estados q_1 , q_{even} y q_{odd} ignoran los símbolos X ya que estos significan que el símbolo visto ya fue procesado.

2. Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \delta)$ una máquina de Turing y $h : \Sigma^* \rightarrow \Sigma^*$ una función. Decimos que M computa h si, para todo $w \in \Sigma^*$, la ejecución $\rho = C_0 \dots C_m$ de M sobre w es finita y $C_m = \vdash q_f h(w)$.

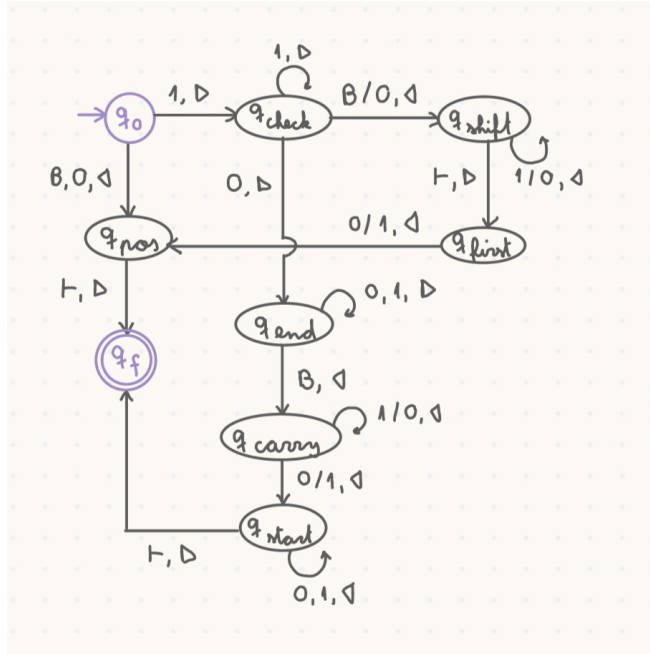
Considere $\Sigma = \{0, 1\}$ y $h : \Sigma^* \rightarrow \Sigma^*$ definida como sigue: si el primer símbolo de $w \in \Sigma^*$ es 1, entonces $h(w) \in \Sigma^*$ es el sucesor de w , siendo ambos vistos como expansiones binarias de enteros positivos; de no ser así, $h(w) := w$. Escriba una descripción formal (alfabeto y diagrama de estados) de una máquina de Turing que compute h .

Solución: Para construir la máquina pedida, podemos separar su comportamiento en 2 casos:

- a) Si el primer símbolo de la palabra de entrada es «0», la palabra se mantiene exactamente igual. Solo debemos preocuparnos de posicionar la cabeza lectora al inicio y aceptar.
- b) Si la palabra de entrada comienza con «1», debemos calcular su sucesor, se identifican 2 casos:
 - 1) La palabra se encuentra compuesta únicamente por símbolos «1»: Al sumar 1, se genera un *carry* desde la unidad hasta el dígito más significativo, por lo que debemos agregar un dígito al número y transformarlo en un «1» seguido de la cantidad de «0»s que corresponda, eg: $111 + 1 = 1000$.
 - 2) La palabra tiene al menos un «0»: Dependiendo de la terminación del número, puede existir un *carry* (si la terminación es 1) o no. Así, si el último dígito corresponde a un «0», basta con cambiarlo por un símbolo «1» (eg: $100 + 1 = 101$). En caso contrario, se genera un *carry* que avanza entre los 1s al final del número, transformándolos en 0s hasta llegar al primer cero visto desde el final de la palabra hacia adelante (en este caso sabemos que no hay overflow, porque sí o sí existe un «0» que ‘impide’ al *carry* seguir avanzando) (eg: $1010111 + 1 = 1011000$).

Una vez calculado el sucesor, debemos preocuparnos de posicionar la cabeza lectora al inicio y aceptar.

A continuación se presenta un diagrama de estados de una máquina de Turing que implementa este procedimiento, y cuyo alfabeto de máquina corresponde a $\Gamma = \{0, 1, \vdash, B\}$:



La máquina comienza en el estado q_0 , y revisa el primer símbolo de la palabra, si este es «0» o si la palabra es vacía («B»), posiciona la cabeza lectora y acepta, al pasar por los estados q_{pos} y q_f estado final. En caso contrario, revisa si la palabra se encuentra compuesta únicamente por «1»s o no. De ser así, al leer el final de la palabra, la cabeza lectora modifica el símbolo «B» leído por un «0», y cambia su estado a q_{shift} , en el cual modifica todos los «1»s vistos por «0» hasta llegar al inicio de la palabra, cambiando su estado a q_{first} , el cual transforma el primer símbolo de la palabra en «1», terminando el cálculo del sucesor. Finalmente, la máquina llega al estado q_{pos} para posicionar su cabeza lectora y luego aceptar.

Si la palabra contiene algún símbolo «0», la máquina transiciona de q_{check} a q_{end} , estado que le permite llegar al final de la palabra. Así, una vez visto el primer símbolo «B» la máquina cambia a q_{carry} estado que calcula el *carry* según lo descrito en (b.2), es decir, de atrás hacia adelante, transformamos todos los «1»s en «0»s, hasta ver un «0» el cual transformamos en «1». Nótese así, que en caso de que la terminación sea «0», la máquina cambia únicamente este símbolo, transformándolo en «1». En cualquier caso, una vez finalizado el cálculo del sucesor, la máquina a cambia su estado a q_{start} , estado que le permite posicionar la cabeza lectora al principio de la palabra, para finalmente aceptar en q_f .

3. Diseñe una máquina de Turing que acepte el siguiente lenguaje:

$$L = \{w \in \{0, 1\}^* \mid \text{la cantidad de 0s de } w \text{ es el doble que su cantidad de 1s}\}$$

Entregue una descripción formal de la máquina (alfabeto y diagrama de estados), junto con un pseudocódigo.

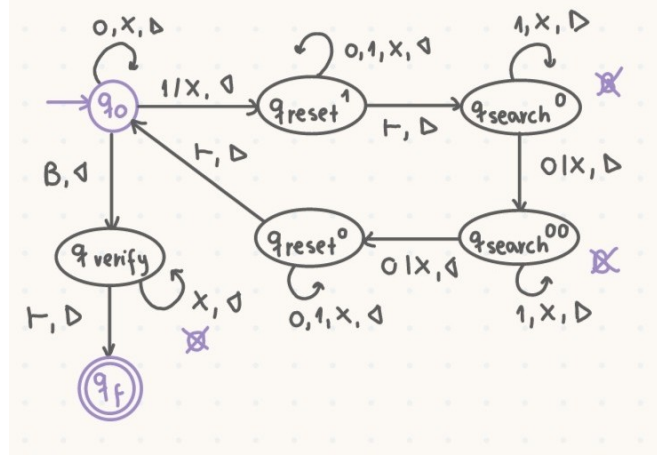
Solución: Para verificar la condición pedida, podemos recorrer la palabra asegurando que por cada símbolo «1» visto existan 2 símbolos «0». Para esto, definimos $\Gamma = \{0, 1, X, \vdash, B\}$ como el alfabeto de la máquina y definimos su comportamiento en base al siguiente pseudocódigo:

1. Mover la cabeza de la máquina hacia la derecha, hasta encontrar un símbolo «1», escribir un símbolo «X» en su lugar.
2. Si en el paso (1) no encontramos un símbolo «1», verificamos que no queden símbolos «0»s en la palabra y **aceptamos**. Si encontramos algún «0», **rechazamos**.
3. Mover la cabeza lectora al inicio de la cinta.

4. Mover la cabeza hacia la derecha hasta encontrar dos «0»s. Cuando los encontramos escribimos un símbolo «X» en su lugar.
5. Si en el paso (4) no encontramos dos «0»s, rechazar. En caso contrario, mover la cabeza al inicio e ir al paso (1).

El diagrama de estados correspondiente es el siguiente:

La máquina comienza en el estado q_0 donde busca un símbolo «1», ignorando todos los demás. Si no lo encuentra (llega al final de la palabra «B»), la máquina transiciona al estado q_{verify} y la cabeza lectora recorre nuevamente la palabra (esta vez en reversa) revisando que se encuentre completamente tachada, es decir, que no queden «0»s sin considerar, de modo que se cumpla la proporción de «0»s y «1»s pedida.



Si es así, la máquina cambia su estado a q_f estado final, y acepta. En caso contrario, rechaza por no existir una transición leyendo «0» desde q_{verify} .

Si la cabeza lectora encuentra un símbolo «1» desde q_0 , la máquina lo tacha, reemplazándolo con «X» y cambia su estado a q_{reset}^1 , en el cual se devuelve la cabeza lectora al inicio de la palabra. Desde ahí, se buscan los 2«0»s necesarios para cumplir la proporción pedida, pasando la máquina por los estados q_{search}^0 y q_{search}^{00} , en búsqueda del primer y segundo «0» respectivamente. Véase que al encontrar un símbolo «0» este es tachado, para evitar revisarlo nuevamente. Nótese también que si en cualquiera de estos estados la cabeza lectora llega al final de la palabra, la máquina rechaza, puesto que no hay transiciones leyendo «B» desde q_{search}^0 y q_{search}^{00} . Finalmente, si se logran encontrar los 2 símbolos «0», la máquina al ser capaz de encontrar la proporción pedida, transiciona al estado q_{reset}^1 , el cual devuelve la cabeza lectora al inicio de la palabra, para repetir el procedimiento.